

Project Management System

Build a web application that enables users to manage projects and tasks.

Users should be able to create projects, organize tasks within projects, track progress, and view overall statistics through a dashboard.

Functional Requirements

1. User Authentication

The application must support:

- User Registration
- User Login
- User Logout

User Fields

- Full Name
- Email Address
- Password

Requirements

- Email addresses must be unique.
- Passwords must never be stored in plain text.
- Authenticated users should remain logged in until logout or token expiration.

2. Project Management

Users should be able to:

- Create a project
- View project details
- Edit a project
- Delete a project
- View all projects they own

Project Fields

- Project Name
- Description
- Status
 - Not Started
 - In Progress
 - Completed

- Start Date
- End Date
- Created Date

3. Task Management

Each project can contain multiple tasks.

Users should be able to:

- Create tasks
- Edit tasks
- Delete tasks
- Mark tasks as completed
- View tasks under a project

Task Fields

- Task Name
- Description
- Priority
 - Low
 - Medium
 - High
- Status
 - Pending
 - In Progress
 - Completed
- Due Date
- Created Date

4. Dashboard

Provide a dashboard displaying:

- Total Projects
- Total Tasks
- Completed Tasks
- Pending Tasks
- Projects In Progress

The dashboard should update based on the authenticated user's data.

5. Search and Filtering

Users should be able to:

- Search projects by name
- Search tasks by name
- Filter projects by status
- Filter tasks by status
- Filter tasks by priority

Technical Requirements

Frontend

Choose one:

- React
- Next.js

Requirements

- Responsive Design
- Proper Component Structure
- Form Validation
- Loading Indicators
- Error Handling
- Clean User Experience

Backend

Choose one:

- Node.js with Express
- NestJS

Requirements

- REST API Architecture
- Proper Route Organization
- Middleware Usage
- Error Handling
- Logging
- Clean Code Structure

Database

Choose one:

- PostgreSQL
- MySQL

Requirements

- Proper Relational Design
- Foreign Key Relationships
- Normalized Database Structure

Security Requirements

Security is an important part of this assessment.

Authentication Security

- Passwords must be hashed using bcrypt or equivalent.
- Plain-text passwords must never be stored.
- Protected APIs must require authentication.

Authorization

Users must only be able to:

- View their own projects
- Modify their own projects
- Delete their own projects
- View their own tasks
- Modify their own tasks

Users must not be able to access data belonging to other users.

Input Validation

Validate all incoming requests.

Examples:

- Required fields
- Invalid email formats
- Invalid date values
- Empty strings
- Invalid enum values

Return appropriate error responses.

API Security

Implement:

- JWT Authentication
- Authentication Middleware
- Protected Routes

Sensitive information must not be exposed in API responses.

Database Security

The application must be protected against SQL Injection.

Use:

- ORM methods
- Query parameterization
- Prepared statements

Avoid raw user input in database queries.

Basic Rate Limiting

Implement rate limiting on authentication endpoints to reduce brute-force attack risks.

Example:

- Limit repeated login attempts from the same IP.

API Expectations

The following endpoints are expected as a minimum.

Authentication

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

Projects

GET /api/projects

GET /api/projects/{id}

POST /api/projects

PUT /api/projects/{id}

DELETE /api/projects/{id}

Tasks

GET /api/tasks

GET /api/tasks/{id}

POST /api/tasks

PUT /api/tasks/{id}

DELETE /api/tasks/{id}

Documentation Requirements

Provide:

- Project Setup Instructions
- Environment Variable Documentation
- Database Setup Instructions
- API Documentation

The application should be easy to run by another developer.

Submission Requirements

Submit the following:

1. GitHub Repository Link
2. Database Schema or ER Diagram
3. API Documentation
4. README File
5. Deployment URL

Bonus Features (Optional)

The following features are optional and will be considered positively during evaluation:

- Docker Support
- Unit Tests
- Integration Tests
- Pagination
- Sorting
- Audit Logs
- Role-Based Access Control
- CI/CD Pipeline
- Application Deployment

Important Notes

- You may use libraries, frameworks, AI-assisted tools, and online resources.
- You should be able to explain your implementation and design decisions during the interview.
- Code readability, maintainability, and security practices are important evaluation criteria.
- The application does not need to be production-ready but should demonstrate good engineering practices.